DESIGN AND IMPLEMENTATION OF WEB SCRAPPING USING BEAUTIFUL SOUP

A

MAJOR PROJECT-II REPORT

Submitted in partial fulfillment of the requirements

for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

By

GROUP NO-35

Manish Singh Shayam	**0187CS211099**
Praful Solanki	**0187CS211119**
Nidhi Rani	**0187CS211108**
Md Hedayatullah	**0187CS211100**

Under the guidance of

Prof. Pankaj Savita

(Assistant Professor)

Department of Computer Science & Engineering

Sagar Institute of Science &Technology (SISTec), Bhopal (M.P)

Approved by AICTE, New Delhi & Govt. of M.P.

Affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

June -2025

Sagar Institute of Science & Technology (SISTec), Bhopal (M.P)

Department of Computer Science & Engineering

CERTIFICATE

We hereby certify that the work which is being presented in the B.Tech. Major Project-II Report entitled **DESIGN AND IMPLEMENTATION OF WEB SCRAPPING USING BEAUTIFUL SOUP,** in partial fulfillment of the requirements for the award of the degree of ***Bachelor of Technology*****,** submitted to the Department of **Computer Science & Engineering**, Sagar Institute of Science & Technology (SISTec)**,** Bhopal (M.P.) is an authentic record of our own work carried out during the period from Jan-2025 to Jun-2025 under the supervision of **Prof. Pankaj Savita.**

The content presented in this project has not been submitted by me for the award of any other degree elsewhere.

Manish Singh	**Praful Solanki**	**Nidhi Rani**	**Md Hedayatullah**
0187CS211099	**0187CS211119**	**0187CS211108**	**0187CS211100**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Prof. Pankaj Savita	**Dr. Amit Kumar Mishra**	**Dr. D.K. Rajoriya**
Project Guide	**HOD, CSE**	**Principal**

ACKNOWLEDGEMENT

We would like to express our sincere thanks to **Dr. D. K. Rajoriya**, **Principal, SISTec and Dr. Swati Saxena, Vice Principal SISTec** Gandhi Nagar, Bhopal for giving us an opportunity to undertake this project.

We also take this opportunity to express a deep sense of gratitude to **Dr. Amit Kumar Mishra, HOD, Department of Computer Science & Engineering** for his kindhearted support

We extend our sincere and heartfelt thanks to our guide, **Prof. Pankaj Savita,** for providing us with the right guidance and advice at crucial junctures and for showing us the right way.

I am thankful to the **Project Coordinator, Prof. Deepti Jain,** who devoted her precious time in giving us the information about various aspects and gave support and guidance at every point of time.

I would like to thank all those people who helped me directly or indirectly to complete my project whenever I found myself in any issue,

TABLE OF CONTENTS

TITLE		PAGE NO.
Abstract		i
List of abbreviations		ii
List of figures		iii
Chapter 1	Introduction	1
	1.1 About Project	1
	1.2 Project Objectives	2
Chapter 2	Software & Hardware Requirements	6
Chapter 3	Problem Description	9
Chapter 4	Literature Survey	12
Chapter 5	Software Requirements Specification	14
	5.1 Functional Requirements	14
	5.2 Non-Functional Requirements	15
Chapter 6	Software Design	17
	6.1 Use Case Diagram	17
	6.2 Er Diagram	17
	6.3 System Architecture	18
	6.4 Database Schema	18
Chapter 7	Implementation And Testing	19
Chapter 8	Deployment	33
Chapter 9	Result and Output Screens	37
Chapter 10	Conclusion and Future Work	38
References		
Project Summary		
Appendix-1: Glossary of Terms		

ABSTRACT

The exponential growth of web-based information presents challenges for efficient data extraction. This project details the design and implementation of a web scraping solution developed to automate the retrieval of structured data from specific online sources. Utilizing the Python programming language and the powerful Beautiful Soup library, the system parses HTML content fetched from target websites. The design focuses on identifying relevant data points within the web page structure and implementing robust logic to navigate the Document Object Model (DOM) for accurate extraction. This documentation covers the architectural choices, the implementation process using Beautiful Soup for parsing and data selection, methods for handling potential variations in web structure, and the transformation of unstructured web data into a usable, structured format. The resulting application demonstrates the effectiveness of Beautiful Soup for automated web data collection, providing a practical tool for data aggregation and analysis.

LIST OF ABBREVIATIONS

ACRONYM	FULL FORM
SDLC	Software Development Life Cycle
SQL	Structured Query Language
HTML	Hyper Text Markup Language
UML	Unified Modeling Language
MVC	Model View Controller
REST	Representational State Transfer
SOAP	Simple Object Access Protocol
RAD	Rapid Application Development
CI/CD	Continuous Integration / Continuous Deployment
API	Application Programming Interface
IDE	Integrated Development Environment
TDD	Test-Driven Development
BDD	Behavior-Driven Development
MVC	Model View Controller

LIST OF FIGURES

FIG. NO.	TITLE	PAGE NO.
6.1	Use Case diagram	17
6.2	ER diagram	17
6.3	System Architecture	18
6.4	Database Schema	18
9.1	Home Page	37
9.2	files	37

CHAPTER-1
INTRODUCTION

1.1 ABOUT PROJECT

The World Wide Web represents an immense and rapidly growing repository of information. However, manually extracting specific data from numerous websites is often tedious, time-consuming, and impractical for large-scale analysis. The Web Scraping project is a Python-based solution designed to automate the process of data extraction from websites. It utilizes the powerful Beautiful Soup library as the core parsing engine, allowing for efficient navigation and manipulation of HTML and XML document structures. This system reduces the need for manual data collection, enables the gathering of large datasets quickly, and facilitates data-driven decision-making across various domains. Inspired by the increasing need for accessible and structured data for analysis, research, and business intelligence, this project addresses the inefficiencies of manual methods, making data extraction more efficient and scalable.

The project's primary goals include:

- **Data Extraction Efficiency:** To automate the retrieval of specific information from target web pages, significantly reducing manual effort and time.
- **Structured Data Output:** To transform unstructured or semi-structured data found within web page HTML into a structured format (e.g., CSV, JSON) suitable for analysis, storage, or further processing.
- **Targeted Information Retrieval:** To accurately identify and extract predefined data elements based on their position and attributes within the HTML structure of designated websites.

The key functionalities include:

- **Web Page Fetching:** Programmatically sending HTTP requests to specified URLs to retrieve the raw HTML content of the target web pages.
- **HTML Parsing:** Utilizing the Beautiful Soup library to parse the fetched HTML content, transforming it into a navigable object structure that represents the Document Object Model (DOM).
- **Data Identification and Extraction:** Implementing logic to search and navigate the parsed HTML structure using Beautiful Soup's methods (e.g., finding elements by tag, class, ID) to locate and extract the desired data points (text, links, attributes).

- **Data Structuring and Output:** Organizing the extracted raw data into a predefined structure, such as lists, dictionaries, or data frames, and saving it to a file (e.g., CSV, JSON) or making it available for other applications.

1.2 PROJECT OBJECTIVE

The primary objective of this project is to design and implement an efficient, automated Web Scraping tool using Python and the Beautiful Soup library. This tool aims to extract specific, targeted data from designated websites and convert it into a structured, usable format. The system seeks to provide a reliable method for gathering web data, reducing manual intervention, and enabling applications such as market research, price comparison, content aggregation, and data analysis. By leveraging Beautiful Soup for robust HTML parsing, the project enhances the efficiency and scalability of web data extraction processes.

- **Automate Data Extraction:** Create a system (script/application) that automatically fetches HTML content from specified URLs and extracts predefined data elements without manual intervention.
- **Ensure Data Accuracy:** Develop parsing logic that precisely identifies and retrieves the intended information from the complex structure of web pages, minimizing errors.
- **Provide Structured Output:** Transform the extracted web data into organized formats like CSV or JSON, making it readily available for analysis, reporting, or database ingestion.
- **Develop Robust Scraping Logic:** Design the scraper to be as resilient as possible to minor variations or common patterns in website layouts, although significant changes will still require updates.

1.3 FUNCTIONALITY

- **URL Input and Management:** The system accepts one or more target URLs as input, either directly, through a configuration file, or via command-line arguments, to specify the web pages to be scraped.
- **HTTP Request Handling:** It utilizes a library like requests in Python to send GET requests to the target URLs, retrieve the HTML source code, and handle basic HTTP responses and potential errors.
- **HTML Parsing with Beautiful Soup:** The core functionality involves using Beautiful Soup to parse the retrieved HTML content. It creates a parse tree that allows for easy searching and navigation of the HTML tags and attributes.
- **Data Navigation and Selection:** The system implements specific logic using Beautiful Soup's

search methods (e.g., find(), find_all(), CSS selectors) to pinpoint the exact HTML elements containing the desired data based on their tags, classes, IDs, or relative positions.

- **Data Extraction and Cleaning:** Once elements are located, the system extracts relevant information, such as the text content, attribute values (href for links, src for images), etc. Basic cleaning, like removing excess whitespace, may also be performed.
- **Structured Data Output:** The extracted and cleaned data is organized into a structured format (e.g., a list of dictionaries, where each dictionary represents an item scraped) and typically saved to a file like CSV or JSON for easy use.

1.4 INTERFACE

The interface for the Web Scraping project primarily involves how the user configures, executes, and receives output from the scraper, rather than a graphical user interface. Key aspects include:

- **Configuration Input:** Users typically define target URLs, data selectors (like specific HTML tags or classes to look for), and output file names either through command-line arguments when running the script, a separate configuration file (e.g., config.ini or config.json), or directly within the script's variables.
- **Execution Feedback:** During runtime, the system may provide feedback to the console, indicating the URLs being processed, the number of items found, progress status, or any errors encountered during fetching or parsing. This logging helps monitor the scraping process.
- **Data Output Format:** The primary 'output interface' is the structured data file generated (e.g., a CSV file with clearly defined columns corresponding to the extracted data fields, or a JSON file with nested objects representing the scraped information). The structure of this output is predefined based on the project's goals.

1.5 DESIGN AND IMPLEMENTATION CONSTRAINTS

- **Website Structure Dependency:** The scraper's logic is tightly coupled to the HTML structure of the target websites. Any changes to the website layout, class names, or element hierarchy by the website owners can break the scraper, requiring code updates.
- **Dynamic Content Loading (JavaScript):** Beautiful Soup primarily parses static HTML received from the server. It cannot inherently handle content loaded or modified dynamically using JavaScript after the initial page load. Scraping such content would require additional tools like Selenium or Playwright.
- **Anti-Scraping Mechanisms:** Target websites may implement measures to detect and block automated scraping, such as CAPTCHAs, IP address rate limiting or blocking, user-agent string filtering, or requiring JavaScript execution. Overcoming these can be complex or impossible.

- **Legal and Ethical Considerations:** Scraping must adhere to the website's robots.txt file, Terms of Service, and copyright laws. Ethical scraping practices dictate avoiding overly aggressive requests that could overload the website's server.
- **Network Reliability and Errors:** The scraper relies on network connectivity. It must be designed to handle potential network errors, timeouts, and different HTTP status codes (e.g., 404 Not Found, 403 Forbidden, 503 Service Unavailable).
- **Maintenance Overhead:** Due to the dependency on website structures, web scrapers often require ongoing maintenance and updates to remain functional as target sites evolve over time.

1.6 ASSUMPTIONS AND DEPENDENCIES

The following are the assumptions and dependencies of the project:

1.6.1 ASSUMPTIONS

- **Target Website Accessibility:** It is assumed that the target websites are publicly accessible over the internet and do not require complex authentication mechanisms not accounted for in the design.
- **Initial Website Structure Stability:** The design assumes that the HTML structure of the target websites is relatively consistent at the time of development and testing, allowing the defined selectors to work correctly.
- **Parsable HTML:** It is assumed that the HTML content of the target pages, while potentially complex, is sufficiently well-formed for Beautiful Soup and its underlying parser (e.g., html.parser, lxml) to process without critical errors.
- **Network Connectivity:** The system assumes the machine executing the scraper has a stable and active internet connection to reach the target websites.

1.6.2 DEPENDENCIES

- **Python Environment:** The project requires a functional Python interpreter (specify version range if applicable, e.g., Python 3.6+) installed on the host system.
- **Core Libraries:** The implementation depends heavily on external Python libraries, primarily BeautifulSoup4 for parsing and usually the requests library for making HTTP requests. These must be installed.
- **HTML Parser:** Beautiful Soup itself depends on an underlying HTML parser library (e.g., Python's built-in html.parser, or the faster lxml, or the lenient html5lib). The chosen parser must be available.

- **Target Website Availability:** The scraper's operation is entirely dependent on the target websites being online, accessible, and serving the expected HTML content.

This project aims to implement an effective tool for automated web data extraction. Currently, gathering data from multiple online sources requires significant manual effort, hindering timely analysis and research. Websites present information in diverse HTML structures, making universal extraction challenging. Manual copying is error-prone and not scalable for large datasets. This project seeks to address these issues by designing and implementing a targeted web scraper using Beautiful Soup, providing a means to automatically collect specific data points efficiently and structure them for immediate use, thereby facilitating access to valuable web information.

CHAPTER-2
SOFTWARE & HARDWARE REQUIREMENTS

2.1 INTRODUCTION

The development, execution, and successful operation of the Web Scraping project using Beautiful Soup rely on a specific set of software tools and libraries, along with standard computing hardware. These components work together to enable the fetching of web content, parsing of HTML structures, extraction of desired data, and storage of the results effectively.

2.2 SOFTWARE REQUIREMENTS

2.2.1 PYTHON PROGRAMMING LANGUAGE: Python serves as the foundational programming language for this project. It is an interpreted, high-level, general-purpose programming language known for its clear syntax and readability. Its extensive standard library and vast ecosystem of third-party packages make it exceptionally well-suited for web scraping tasks, string manipulation, network communication, and data handling. Python's cross-platform nature (running on Windows, macOS, Linux) ensures broad compatibility. The project utilizes Python 3.8] for its scripting capabilities, facilitating the sequential execution of fetching, parsing, and data extraction steps. User-written code involves defining functions and logic to manage URLs, interact with web pages, process data using libraries, and handle potential errors.

2.2.2 BEAUTIFUL SOUP 4 (BS4) LIBRARY: Beautiful Soup is a crucial Python library designed for pulling data out of HTML and XML files. It works with your chosen parser to provide idiomatic ways of navigating, searching, and modifying the parse tree. Its primary function within this project is to take the raw HTML content fetched from target websites and transform it into a complex tree of Python objects. This allows developers to easily search for specific HTML tags based on their names, attributes (like class, id, href), or textual content. Beautiful Soup gracefully handles malformed or imperfect HTML, making it robust for dealing with real-world web pages. It significantly simplifies the task of extracting specific data points buried within nested HTML structures.

2.2.3 REQUESTS LIBRARY: The requests library is a standard Python library for making HTTP requests. Its primary role in this project is to fetch the actual HTML content from the target URLs provided to the scraper. It allows for sending various types of HTTP requests (primarily GET requests for scraping), managing URL parameters, handling cookies and sessions (if needed for navigating websites), inspecting response headers, and checking status codes to ensure successful page retrieval.

Its simple and elegant API makes interacting with web services and retrieving web page source code straightforward and efficient, abstracting away the complexities of lower-level HTTP communication.

2.2.4 HTML PARSER (E.G., LXML OR HTML.PARSER): Beautiful Soup itself doesn't parse HTML directly; it relies on an underlying parser library. Common choices include:

- **lxml:** A fast and robust parser written in C (with Python bindings). It's generally preferred for speed and its ability to handle broken HTML well, but requires installation of external C dependencies on some systems.
- **html.parser:** Included in Python's standard library. It's reasonably lenient with malformed HTML and requires no external installation, making it a convenient choice, though typically slower than lxml.
- **html5lib:** Aims to parse HTML exactly as a web browser does, making it extremely lenient but also the slowest option. The choice of parser "This project primarily utilizes the lxml parser for its speed and robustness." impacts performance and how certain types of invalid HTML might be handled during the parsing stage facilitated by Beautiful Soup.

2.2.5 INTEGRATED DEVELOPMENT ENVIRONMENT (IDE) / TEXT EDITOR (OPTIONAL BUT RECOMMENDED): While not strictly required to run the script, development was facilitated using a code editor or IDE such as Visual Studio Code, PyCharm, Sublime Text, Jupyter Notebook. These environments provide features like syntax highlighting for Python and HTML, code completion, debugging tools, integrated terminals for running scripts, and version control integration (like Git), significantly improving developer productivity and code quality during the implementation and testing phases.

2.2.6 Operating System: The web scraping scripts and required libraries (Python, Beautiful Soup, Requests) are designed to be cross-platform. Development and testing for this project were conducted primarily on Windows 10, macOS Monterey, Ubuntu Linux 20.04. However, the system is expected to function correctly on other major operating systems where Python and the necessary libraries can be installed.

2.3 HARDWARE REQUIREMENTS

2.3.1 COMPUTER SYSTEM: The web scraping application does not require specialized hardware; it runs on a standard personal computer (desktop or laptop). Key considerations include:

- **Processor (CPU):** A modern multi-core processor is sufficient for running Python scripts and handling the parsing load. Performance may vary depending on the complexity of the websites and the volume of data being processed.
- **Memory (RAM):** A minimum of 4 GB or 8 GB RAM is recommended. Sufficient memory is needed to run the operating system, the Python interpreter, potentially a web browser for inspection, and to hold the parsed HTML documents and extracted data in memory, especially when dealing with large pages or datasets.
- **Storage:** Adequate free disk space is required for installing Python, the necessary libraries, storing the project source code, and saving the output data files (e.g., CSV, JSON), which could potentially become large depending on the scale of scraping.
- **Operating System:** A compatible operating system (Windows, macOS, or Linux) capable of running the specified software requirements.

2.3.2 NETWORK CONNECTION: A stable and active internet connection is a critical hardware requirement. The scraper needs to communicate over the network to send HTTP requests to the target websites and receive their HTML content. The reliability and speed of the internet connection directly impact the scraper's performance, determining how quickly pages can be fetched and influencing the success rate, especially when scraping multiple pages or websites. Unstable connections can lead to timeouts and incomplete data extraction.

CHAPTER-3
PROBLEM DESCRIPTION

3.1 OVERVIEW

The internet represents an unprecedented repository of data, yet accessing this information systematically remains a significant hurdle. While web browsers provide a human-readable interface to navigate this data one page at a time, this method is inadequate for gathering and processing large amounts of information quickly. Manually collecting data from websites is inherently slow, error-prone, and does not scale to meet the demands of modern data analysis, market research, or competitive intelligence. The inability to efficiently harness web data means missing out on a huge range of possibilities that automated techniques unlock. This project addresses the need for automated data extraction by focusing on web scraping: the practice of writing automated programs to query web servers, request data (typically HTML), and parse that data to extract needed information. It aims to provide a more efficient, scalable, and reliable alternative to manual methods.

3.1.1 THE CHALLENGES OF TRADITIONAL WEB DATA COLLECTION METHODS

Relying solely on manual Browse and copy-pasting for web data collection presents substantial challenges:

- **Lack of Scalability:** Humans can only process a limited number of pages. Manually collecting data from thousands or millions of pages, or tracking frequent updates, is practically impossible.
- **Time and Cost Inefficiency:** Manual collection is extremely labor-intensive, translating directly into high costs and significant time delays in obtaining data. Tasks that could take minutes for a scraper might take days or weeks manually.
- **Error Proneness:** Manual data entry is highly susceptible to human errors, such as typos, omissions, or inconsistencies in formatting, which compromises data quality.
- **Inability to Access "Deep Web" Data:** Manual Browse often fails to capture data generated dynamically or requiring specific inputs (like flight search results), which automated scripts can potentially access.
- **Data Structure Issues:** Data copied manually often lacks consistent structure, requiring significant further effort to clean and organize before it can be analyzed or used effectively.

These limitations mean that valuable data remains inaccessible or underutilized, hindering analysis and decision-making.

3.1.2 THE IMPORTANCE OF AUTOMATED WEB SCRAPING

Automated web scraping provides a powerful solution to the inefficiencies of manual data collection. By writing scripts, often referred to as bots or scrapers, we can systematically interact with websites to gather specific information. This automated approach is crucial for several reasons:

- **Speed and Efficiency:** Web scrapers can gather data from numerous pages far faster than any human, enabling the collection of large datasets in a short time.
- **Scalability:** Automated scripts can be scaled to handle vast numbers of pages and websites, limited primarily by computing resources and politeness considerations towards target servers, rather than human endurance.
- **Data Consistency and Structure:** Scrapers extract data based on predefined rules, ensuring consistency in format and structure, making the data readily usable for analysis or database storage.
- **Accessing Hidden or Dynamic Data:** Well-designed scrapers can interact with forms, handle logins (Chapter 10 in the book) [2], and potentially execute JavaScript (Chapter 11) [2] or query APIs (Chapter 12) [2], accessing data unavailable through simple Browse or basic scraping.
- **Enabling New Applications:** Access to large-scale, structured web data enables applications ranging from market forecasting and competitive analysis to machine learning model training and academic research.

While APIs are preferable for data access when available and suitable, they often do not exist, lack necessary data fields, have restrictive rate limits, or are simply not provided for valuable or protected data. In such cases, web scraping becomes the necessary tool to acquire the required information.

3.1.3 KEY FEATURES AND FUNCTIONALITY (of the Proposed Web Scraper) This project, focusing on implementation using Python and Beautiful Soup, aims to provide key functionalities that directly address the problems of manual data collection:

- **Automated HTTP Interaction:** The system programmatically sends requests to web servers to fetch page content, replicating the initial step of a browser but under script control.
- **Targeted HTML Parsing:** Utilizing Beautiful Soup, the system intelligently parses the complex and often messy HTML structure of web pages. It allows for precise targeting of data using tags, attributes, and text content, navigating the document tree effectively.
- **Structured Data Extraction:** The scraper identifies and extracts the specific required data points, transforming them from their embedded HTML context into a structured format (e.g., Python dictionaries, lists) suitable for output.

- **Reliable Connection Handling:** Incorporates mechanisms to handle common web connection issues, such as page-not-found errors or server timeouts, making the scraping process more robust (as discussed conceptually regarding reliable connections in Chapter 1 of the book) [2].
- **Foundation for Scalability:** While this project focuses on Beautiful Soup, the core principles establish a foundation that could be extended with more advanced crawling techniques (like those in Chapters 3 & 4 of the book) [2] or frameworks like Scrapy (Chapter 5) [2] for larger-scale tasks.

By implementing these features, the project delivers a practical demonstration of web scraping's power to efficiently collect structured data from the web, overcoming the significant limitations of traditional manual methods.

CHAPTER-4
LITERATURE SURVEY

Web scraping, the automated process of extracting data from websites, has become an essential technique in the data science toolkit, addressing the challenge of accessing the vast and often unstructured information available on the internet. As described by Ryan Mitchell in "Web Scraping with Python, 2nd Edition[2]," this field involves writing programs to query web servers, retrieve responses (typically HTML), and parse this data to isolate and collect specific information. The necessity for web scraping arises frequently because formal APIs may not exist, may be inadequate, or may intentionally restrict access to the desired data. Literature in the field, including Mitchell's comprehensive guide, covers a wide range of techniques from basic page fetching to sophisticated interactions with modern web applications.

The foundational mechanics of web scraping, as detailed in Part I of Mitchell's book, involve using programming languages like Python along with libraries such as requests to handle HTTP communication with web servers and Beautiful Soup to parse the returned HTML. A significant challenge universally acknowledged in the literature is dealing with the inherent messiness and variability of web content. HTML is often poorly structured or changes frequently, requiring robust parsing techniques. Beautiful Soup is frequently highlighted for its ability to navigate and search these complex or malformed HTML/XML documents effectively, making it a cornerstone tool for many scraping tasks focused on static content retrieval.

Beyond basic HTML parsing, the field has evolved significantly to tackle the complexities of the modern web. Literature explores advanced crawling techniques and frameworks, with Scrapy being a prominent example discussed in Mitchell's work (Chapter 5)[2] for building scalable and efficient web spiders capable of navigating entire websites. Handling dynamic content loaded via JavaScript presents another major challenge, often requiring tools like Selenium or Playwright (covered conceptually in Chapter 11)[2] that can control a web browser to render pages fully before extraction. Furthermore, interacting with websites often necessitates navigating through forms, managing login sessions (Chapter 10)[2], and even deciphering and utilizing underlying APIs when possible (Chapter 12) [2], all topics covered extensively in contemporary web scraping literature.

A critical aspect emphasized throughout the literature, including Mitchell's book, is data processing *after* extraction. Raw scraped data is often noisy, poorly formatted, or embedded within text or even images. Techniques for cleaning and normalizing data (Chapter 8) [2], reading and extracting

information from various document formats like PDF or DOCX (Chapter 7) [2], applying natural language processing to understand textual content (Chapter 9) [2], and even using optical character recognition (OCR) to extract text from images (Chapter 13) [2] are integral parts of a complete web scraping workflow. Effective storage strategies for the collected data are also a key consideration (Chapter 6) [2].

Ethical and legal considerations are paramount in web scraping literature. Mitchell dedicates significant attention (Chapter 14) [2] to navigating the complex landscape of "scraping traps," bot blockers, and the legal implications surrounding website Terms of Service and copyright law. Respecting robots.txt directives, implementing appropriate delays (rate limiting), and managing browser headers (like User-Agents) to minimize server load and avoid detection are presented as crucial practices for responsible scraping. The literature acknowledges an ongoing "arms race" between website administrators seeking to protect their data and scrapers developing more sophisticated techniques.

The applications of web scraping documented in the literature are diverse and impactful. They range from academic research gathering large datasets, to e-commerce businesses monitoring competitor pricing and product availability, to news organizations aggregating information, financial institutions tracking market sentiment, and even using scrapers for automated website testing (Chapter 15) [2]. The ability to systematically collect and structure web data fuels countless data analysis, machine learning, and business intelligence initiatives.

This project, focusing on the design and implementation of web scraping using Beautiful Soup, situates itself within this broader context. It leverages the well-established strengths of Beautiful Soup for parsing HTML, as detailed extensively by Mitchell, tackling the common challenge of extracting structured data from static web pages. While acknowledging the limitations of Beautiful Soup in handling dynamically loaded content compared to browser automation tools or API interaction methods discussed elsewhere in the literature, this project provides a practical implementation of core scraping principles applicable to a vast number of websites.

CHAPTER-5
SOFTWARE REQUIREMENT SPECIFICATION

5.1 FUNCTIONAL REQUIREMENTS

5.1.1 URL INPUT AND PROCESSING

The system must accept a list of target URLs as input (e.g., via a configuration file, command-line argument, or hardcoded list). The system should perform a basic validation check to ensure inputs resemble valid URLs.

5.1.2 WEB PAGE FETCHING

The system must send HTTP GET requests to each valid target URL. The system must retrieve the complete HTML content returned by the web server for successful requests (HTTP status 2xx). The system must detect and log errors for unsuccessful requests (e.g., 4xx client errors, 5xx server errors).

5.1.3 HTML PARSING

The system must utilize the Beautiful Soup library to parse the retrieved HTML content. The parser must effectively transform the HTML text into a navigable object structure (parse tree). The system must be able to handle common variations and potential minor errors in HTML structure gracefully, leveraging Beautiful Soup's tolerance.

5.1.4 DATA IDENTIFICATION AND EXTRACTION

The system must identify specific target data elements within the parsed HTML based on predefined selectors (e.g., combinations of HTML tags, CSS classes, IDs, attributes). The system must extract the desired information from these elements, such as inner text, specific attribute values (href, src, etc.), or the element's presence. The system must handle scenarios where expected elements are not found on a page (e.g., log a warning and continue processing, or store a null/empty value).

5.1.5 DATA STRUCTURING

The system must aggregate the extracted data points for each processed item (e.g., for each product on a page, or each article) into a consistent structure (e.g., a Python dictionary or object). The system should perform basic data cleaning operations as defined (e.g., stripping leading/trailing whitespace from extracted text).

5.1.6 DATA OUTPUT/STORAGE

The system must compile the structured data from all processed URLs into a final dataset. The system must save this dataset to a persistent file in a specified format (e.g., CSV, JSON). The structure of the output file must match the predefined format (e.g., correct headers and columns in a CSV file).

5.1.7 ERROR HANDLING AND LOGGING

The system must implement basic error handling for network issues (e.g., connection timeouts). The system must log key operational information, including the start and end of the scraping process, URLs being processed, number of items extracted, and the location of the output file. The system must log any significant errors encountered during fetching, parsing, or data extraction that prevent successful processing of a URL or data item.

5.2 NON-FUNCTIONAL REQUIREMENTS

- **Performance:** The system should process the target URLs and extract data within a reasonable timeframe relative to the number of pages. Crucially, it must incorporate configurable delays between consecutive HTTP requests to the same server (e.g., 1-5 seconds) to avoid causing excessive load ("politeness").

- **Reliability:** The system should operate reliably for the target websites under the assumption that their structure does not change drastically. It must handle common, transient network errors gracefully (e.g., log and continue or retry). Failures in processing one URL should not typically halt the entire scraping process for other URLs.

- **Usability:** The process for configuring the scraper (specifying target URLs, selectors, output file) and executing it should be clear and straightforward for a user with basic technical knowledge. Log outputs should be informative enough to understand the scraper's progress and diagnose basic issues.

- **Maintainability:** The source code must be well-organized, readable, and include comments explaining complex logic. Selectors used for identifying data should be defined in an easily accessible and modifiable location within the code or a configuration file to simplify updates when target website structures change.

- **Security & Ethics:** The scraper must operate ethically. It should not attempt to access restricted areas or overload servers. It must not be used to collect sensitive personal data unless explicit

consent and compliance with privacy regulations (like GDPR) are ensured. If possible, it should identify itself via a custom User-Agent string indicating it is a bot. Adherence to robots.txt is recommended for responsible scraping (though may require additional library support).

- **Scalability (Script Level):** The script should be capable of handling input lists containing tens or hundreds of URLs without significant degradation in stability (though overall time will increase). (Note: Large-scale scraping of thousands/millions of pages typically requires more advanced frameworks like Scrapy).

CHAPTER-6
SOFTWARE DESIGN

6.1 USE CASE DIAGRAM

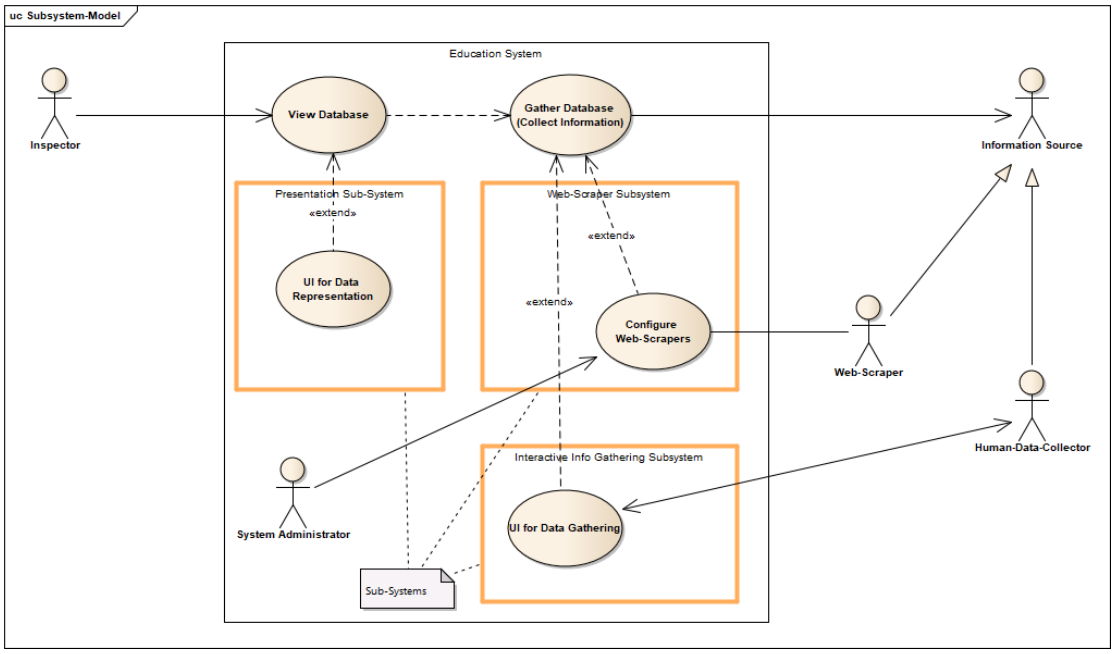


Figure 6.1: Use case Diagram

6.2 ER DIAGRAM

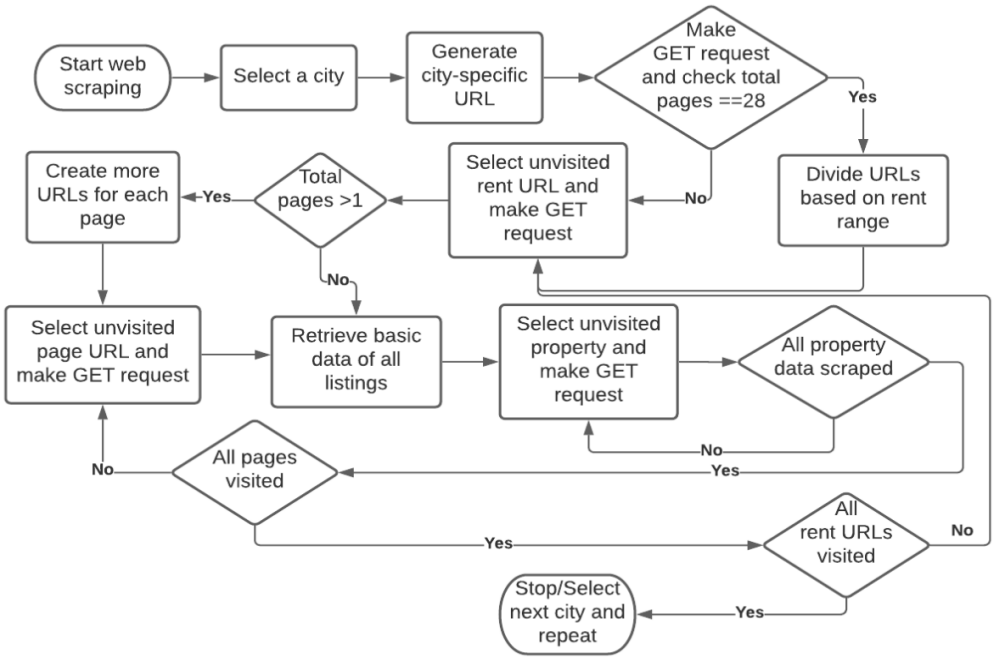


Figure 6.2: ER Diagram

6.3 SYSTEM ARCHITECTURE

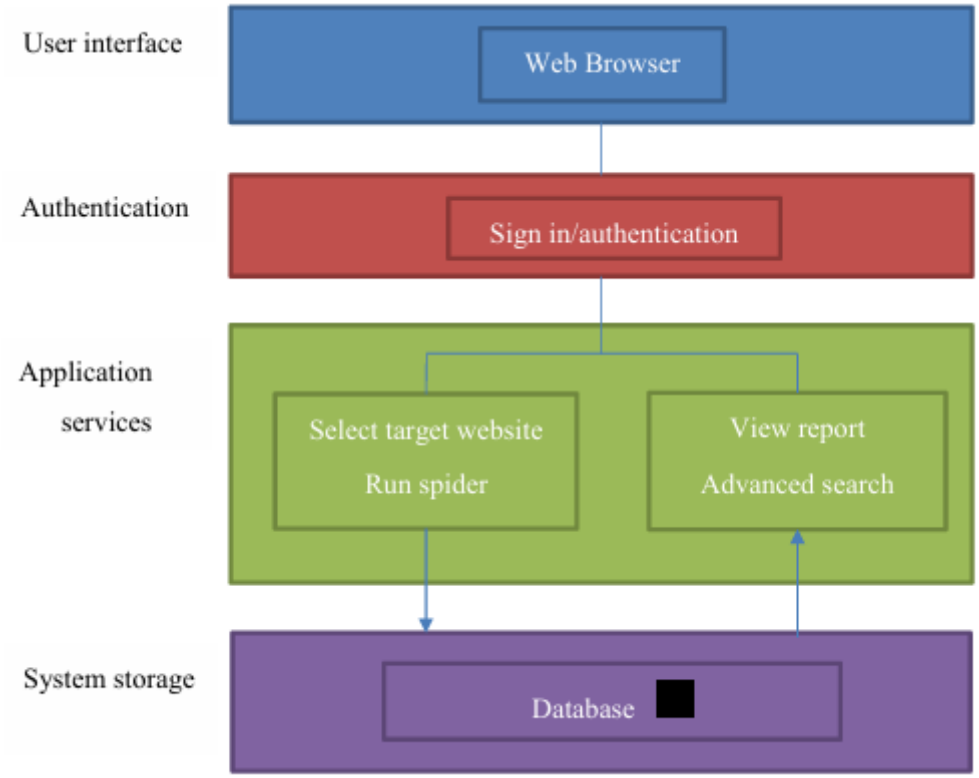


Figure 6.3: System Architecture

6.4 DATABASE SCHEMA

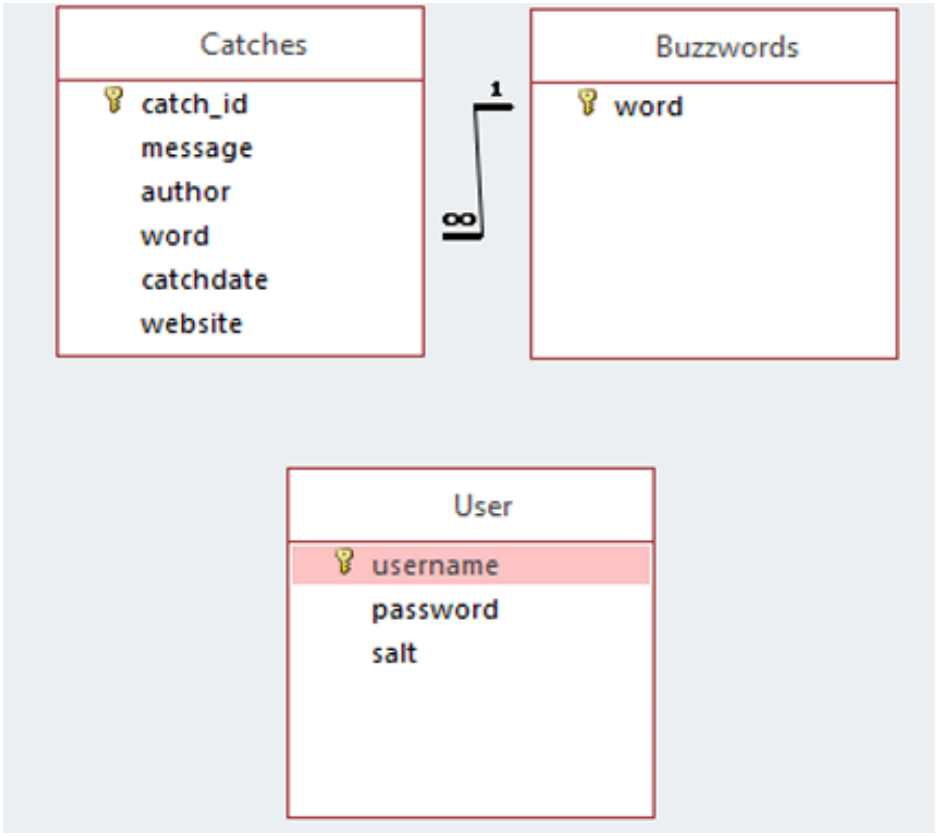


Figure 6.4: Database Schema

CHAPTER-7
IMPLEMENTATION AND TESTING

7.1 DEVELOPMENT APPROACH

The approach taken during the development of the web application was an incremental one. In this approach each segment was completed before moving on to another. The first segment was building the first spider, focusing on a local page taken from reddit.com, where an understanding of how Scrapy works was gained. After this, the database was developed, and the spider was adapted to write its findings to the database. Once this was completed, work began on the web application, where Flask was set up and the first webpages were made; index.html and layout.html. The layout page is a page that never loads by itself, as it provides the layout of the parts of the webpages that do not change between pages. An example of this is the menu, or navbar, at the top of each page, which remains constant no matter which page the application loads. After this, the spider(url) method was created to allow the spider to be activated from within the web application. The report.html page was then created to display the findings of the spider in an easy-to-read fashion. A second spider was then created, this time targeted towards a local page taken from 4chan.org. Lastly, the advanced search was developed to allow a user to send queries to the database and generate custom reports.

The tools and languages used to develop this system were as follows:

• Backend: Python and its libraries; Scrapy, Flask and SQLAlchemy

• Frontend: HTML, CSS, jQuery, and Twitter Bootstrap

These technologies were chosen for two reasons. The first is due to already having experience in using these languages and libraries for developing web applications, with the exception of the Scrapy library, and the second is that Scrapy is a Python library, so the application also had to be built using Python. As Scrapy was the only unfamiliar tool, time was spent learning how to use the framework at the beginning of development before any work began on the other, more familiar components.

7.2 IMPLEMENTATION

7.2.1 DATABASE: The database, called scrape.db, was developed using SQLAlchemy, an object relational mapper, ORM, in Python. This uses classes to create database tables, as shown with the Buzzwords class below.

class Buzzwords(Base):

```
__tablename__ = 'buzzwords'
word = Column(Text, primary_key=True)
@property
def serialise(self):
    return {
        'word': self.word
    }
```

This creates a table in the database called “buzzwords”, with a “word” column being the primary key and only field. The serialise(self) method allows the data to be accessed and displayed by the web application. The Catches class creates a table in the database containing a catch_id as the primary key, a scraped message, the author, the buzzword as a foreign key to the Buzzwords class, or table, the catchdate, and the website.

```
class Catches(Base):
    __tablename__ = 'catches'
    catch_id = Column(Integer, primary_key=True)
    message = Column(Text)
    author = Column(Text)
    word = Column(Text, ForeignKey('buzzwords.word'))
    buzzwords = relationship(Buzzwords)
    catchdate = Column(Text)
    website = Column(Text)
    @property
    def serialise(self):
        return {
            'catch_id': self.catch_id,
```

```
        'message': self.message,
        'author': self.author,
        'word': self.word,
        'catchdate': self.catchdate,
        'website': self.website
    }
```

The catchdate field is stored as a Text field rather than a DateTime because a format of “YYYY-MM-DD hh-mm-ss” was desired and DateTime fields can cause issues when attempting to display the field in a web application.

The User class creates a table with a username, password and salt. This class does not require a serialise method, as user information will not be displayed at any point in the application for security concerns.

```
class User(Base):
    __tablename__ = 'user'
    username = Column(String(20), primary_key=True)
    password = Column(String(20), nullable=False)
    salt = Column(LargeBinary)
```

The database is populated with buzzwords prior to use to ensure that the spiders can function as intended.

```
words = ['automatic', 'bottom', 'bottom bitch', 'branding', 'caught a case', 'choosing up',
        'circuit', 'coercion', 'commercial sex act', 'cousin-in-law', 'cousin in law', 'daddy',
        'date', 'exit fee', 'facilitator', 'family', 'folks', 'finesse pimp', 'romeo pimp',
        'force', 'fraud', 'gorilla pimp', 'guerilla pimp', 'head cut', 'human smuggling',
        'human traffick', 'in-pocket', 'in pocket', 'john', 'trick', 'kiddie stroll', 'loose bitch',
        'lot lizard', 'madam', 'out of pocket', 'pimp', 'pimp circle', 'pimp partner', 'quota',
```

```
    'eyeballing', 'renegade', 'seasoning', 'serving a pimp', 'squaring up', 'stable',
    'the game', 'the life', 'track', 'stroll', 'blade', 'trade up', 'trade down', 'traficker',
    'turn out', 'the wire', 'wifey', 'wife-in-law', 'wife in law', 'sister wife']
for word in words:
  buzz = Buzzwords(word=word)
  session.add(buzz)
session.commit()
```

A list of words relating to sex trafficking is created. This list is then looped through and each individual word, or phrase, is added to the Buzzwords table.

A user is then created with the username “admin” and the password “admin123”. A random 8-byte string is create for use as a salt. The password is then combined with the salt and they are hashed using the SHA-256 hash function. This is then stored as the password for the user and the record is added to the database.

```
password = b"admin123"
salt = urandom(8)
hash_object = hashlib.sha256(password.encode() + salt)
admin = User(username=u"admin", password=hash_object.hexdigest(), salt=salt)
session.add(admin)
session.commit()
```

7.2.2 WEB APPLICATION

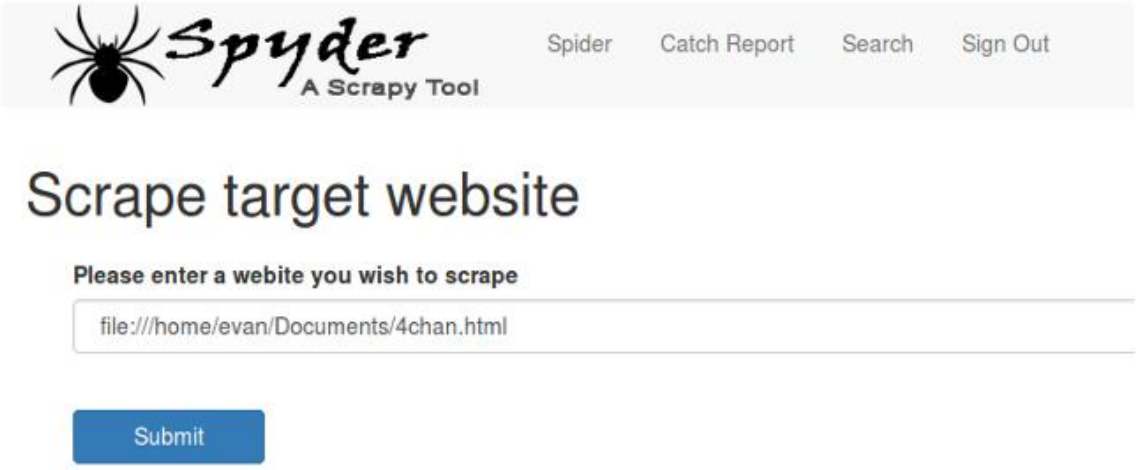
The web application was developed using the Flask framework for Python in the backend, and HTML pages in the frontend. For all HTML pages, see Appendix B.

The main() method renders the index.html page if the user is signed in. The route is the path in the URL bar of a browser. For example, to reach the index page, “localhost:5000/” is entered, and to reach the sign in page, “localhost:5000/signin/” is entered.

```
# open the index page
```

```
@app.route('/', methods=['GET', 'POST'])
def main():
    if g.user:  # check for user session
        if request.method == 'POST':  # webform submission
            url = request.form['site']
            url = url.replace("/", "%2f")
            return redirect(url_for('spider', url=url))  # run the spider
        return render_template('index.html')
    return redirect(url_for('signin'))
```

This method checks to see if a user is signed in by looking for an active session. If the user submits the form on the index.html page, the entered website is sent to the spider to be scraped. Shown below is the index.html pag



The spider(url) method handles the execution of the spiders. Due to the limitations with Scrapy spiders discussed previously, separate spiders are used, therefore, there are checks to see what URL was entered and the corresponding spider is executed.

```
# run the spider
@app.route('/run-spider/<url>')
def spider(url):
```

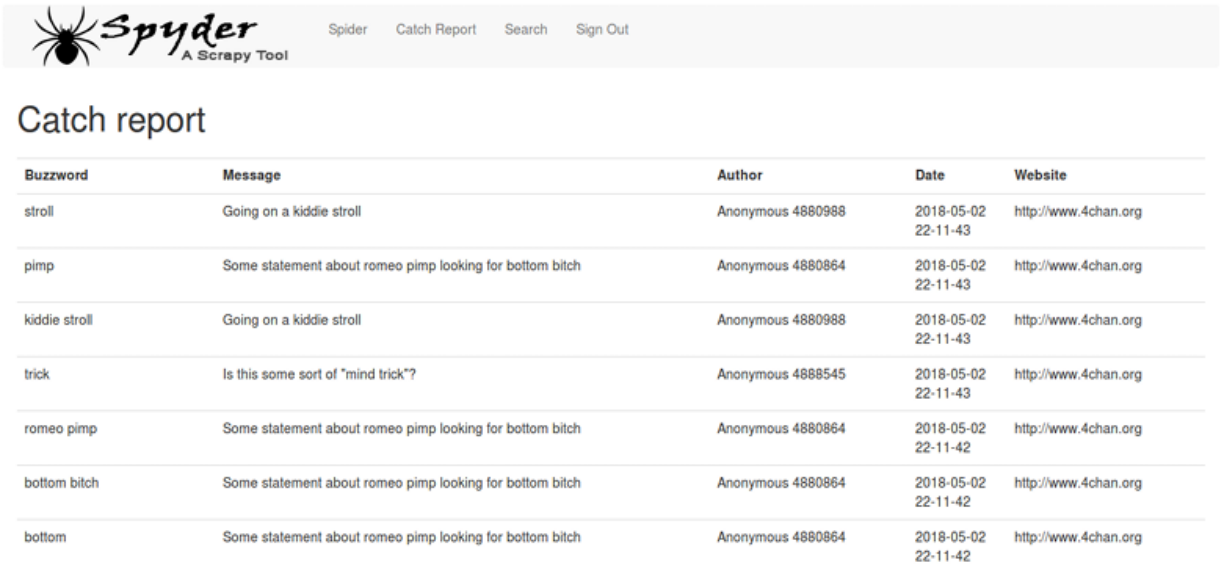
```
    if g.user:  # check for user session
        if 'reddit' in url:
            os.system('scrapy crawl reddit')  # run command
        elif '4chan' in url:
            os.system('scrapy crawl 4chan')
        else:
            return redirect(url_for('main'))
        # ...and so on...
        return render_template('spider.html')
    return redirect(url_for('signin'))
```

Shown below is the spider.html page displaying a message to the user, telling them that the spider is active.

The report() method returns all Catches and displays them in a table in the report.html page.

```
# display report
@app.route('/report/')
def report():
    if g.user:  # check for user session
        query = dbsession.query(Catches)
        return render_template('report.html', query=([catch.serialise for catch in query]))
    return redirect(url_for('signin'))
```

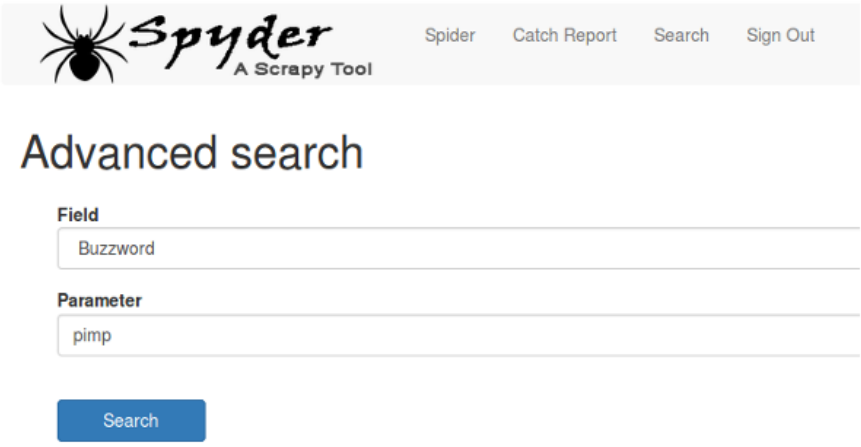
Shown below is the general report generated in report.html by this method.



The advancedSearch() method allows users to search the database for specific terms. Due to this, the method gets the values in the webform in the advanced.html page, and passes them as parameters into the advanceReport(table,field,param) method.

```
@app.route('/report/search/', methods=['GET', 'POST'])
def advancedSearch():
    if g.user:  # check for user session
        if request.method == 'POST':
            if request.form['field']:
                field = request.form['field']
            if request.form['param']:
                param = request.form['param']
                param = param.replace("/", "%2f")
            elif not request.form['param']:
                param = "%"
            return redirect(url_for('advancedReport', table="catches", field=field, param=param))
        return render_template('advanced.html')
    return redirect(url_for('signin'))
```

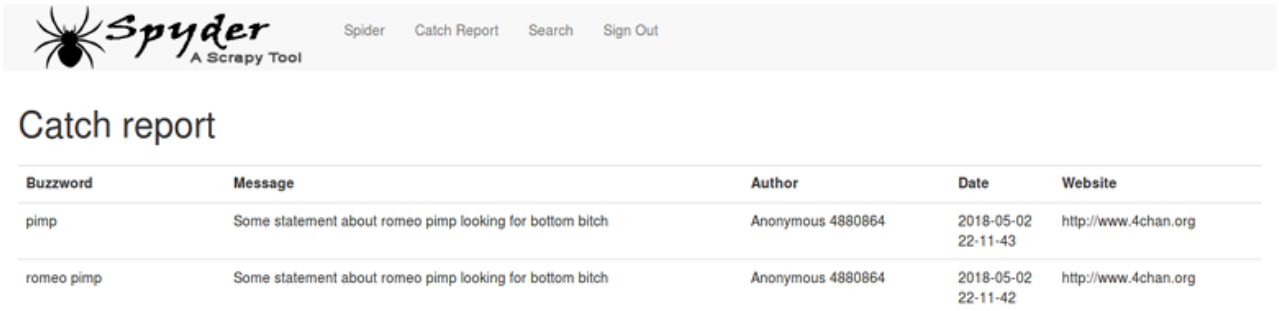
Shown below is the advanced.html page with fields allowing the user to enter data to create custom reports.



The advanced Report(table,field,param) method uses the parameters received from the advanced Search() method to generate a custom query. The results of this query are then added to a dictionary, which is then passed into the report.html page for display.

```
@app.route('/report/<table>?<field>=<param>')
def advancedReport(table, field, param):
    if g.user:  # check for user session
        param = param.replace("%2f", "/")
        query = dbsession.execute(
            "select * from {0} where {0}.{1} like '%{2}%'".format(table, field, param))
        rows = query.fetchall()
        result = []
        for row in rows:
            result.append(dict(row))
        return render_template('report.html', table=table, field=field, param=param, query=result)
    return redirect(url_for('signin'))
```

Shown below is the custom report generated in report.html by this method.



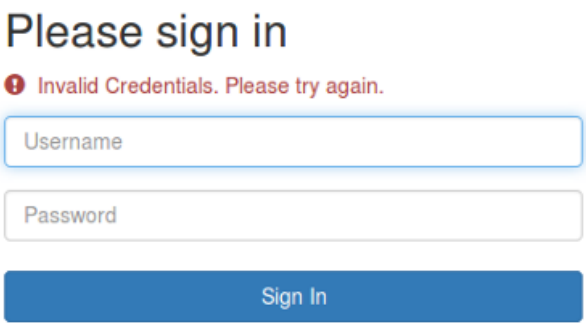
The signin() method deals with the authentication of a user. The entered username and password is checked for validity in the validate(username,password) method by comparing the entered username to the username stored in the database.

To check the password, the check_password (hashed_password,user_password,salt) method is called, which uses a SHA-256 hash function to generate a digest and compare it with the password digest in the database. If the check is unsuccessful, an error message is passed into the signin.html page to be displayed.

```
# compare password with database
def check_password(hashed_password, user_password, salt):
    return hashed_password == hashlib.sha256((user_password.encode()) + salt).hexdigest()
# compare credentials with database
def validate(username, password):
    completion = False
    users = dbsession.query(User)
    for user in users:
        if user.username == username:
            completion = check_password(user.password, password, user.salt)
    return completion
# sign in user
@app.route('/signin/', methods=['GET', 'POST'])
def signin():
    error = None
```

```
    if request.method == 'POST':
        session.pop('user', None)
        uname = request.form['username']
        pword = request.form['password']
        completion = validate(uname, pword)
        if completion == True:
            session['user'] = uname  # create user session
            return redirect(url_for('main'))
        else:
            error = 'Invalid Credentials. Please try again.'
    return render_template('signin.html', error=error)
```

Shown below is the signin.html page, first with no error message, then with the error message displayed during a failed sign in attempt.



The signout() method deletes the active user session, signing the user out of the application.

```
# delete session
@app.route('/signout/')
def signout():
    session.pop('user', None)
    return redirect(url_for('main'))
```

The before_request() method is a Flask method that is called before each request and checks to see if there is an active session.

```
# create user session

@app.before_request

def before_request():

    g.user = None

    if 'user' in session:

        g.user = session['user']
```

7.3 TESTING

For testing this web application, unit testing was carried out using Python's included unit testing framework.

To begin testing a Flask web application, a setup(self) method and a teardown(self) must be created. The setup(self) method creates the necessary connections to the database and sets configurations for use in automated testing. The teardown(self) method closes these connections and unlinks from the database.

```
def setUp(self):

    self.db_fd, app.app.config['DATABASE'] = tempfile.mkstemp()

    app.app.config['DEBUG'] = False

    app.app.config['TESTING'] = True

    app.app.config['SERVER_NAME'] = 'myapp.dev:5000'

    app.app.config['SECRET_KEY'] = 'secret'

    self.app = app.app.test_client()

def tearDown(self):

    os.close(self.db_fd)

    os.unlink(app.app.config['DATABASE'])
```

The first test focuses on user authentication. Three sets of usernames and passwords are tested:

• admin, admin123 (valid, valid)

• adminx, admin123 (invalid, valid)

• admin, default (valid, invalid)

These credentials are then passed into the signin() method to check for validity. The rv.data calls look for text in the rendered webpages.

```
def signin(self, username, password):
    with app.app.app_context():
        return self.app.post(url_for('signin'), data=dict(
            username=username,
            password=password
        ), follow_redirects=True)
def signout(self):
    with app.app.app_context():
        return self.app.get(url_for('signout'), follow_redirects=True)
def test_1_signin_signout(self):
    with app.app.app_context():
        rv = self.signin('admin', 'admin123')
        assert b'Scrape target website' in rv.data
        rv = self.signout()
        assert b'Please sign in' in rv.data
        rv = self.signin('adminx', 'admin123')
        assert b'Invalid Credentials. Please try again.' in rv.data
        rv = self.signin('admin', 'defaultx')
        assert b'Invalid Credentials. Please try again.' in rv.data
```

The second test signs the user out and attempts to access the main() method, asserting that the signin.html page should open instead. The test then signs the user in with valid credentials and attempts to access the main() method, asserting that text from the index.html page should be returned.

```
def test_2_index(self):
    with app.app.app_context():
        self.signout()
        rv = self.app.get(url_for('main'), follow_redirects=True)
        assert b'Please sign in' in rv.data
        self.signin('admin', 'admin123')
        rv = self.app.get(url_for('main'))
        assert b'Scrape target website' in rv.data
```

The third test signs the user out and attempts to access the spider, asserting that the signin.html page should open instead. The test then signs the user in with valid credentials and passes three URLs into the spider(url) method:

- reddit.com (valid)
- 4chan.org (valid)
- invalidsite.com (invalid)

The first two should execute the corresponding spiders, while the third should not execute any spider and should redirect to the index.html page.

```
def test_3_spider(self):
    with app.app.app_context():
        self.signout()
        url = "reddit.com"
        rv = self.app.get(url_for('spider', url=url), follow_redirects=True)
        assert b'Please sign in' in rv.data
        self.signin('admin', 'admin123')
        url = "reddit.com"
        rv = self.app.get(url_for('spider', url=url))
        assert b'Please wait while the spider scrapes the target website...' in rv.data
        url = "4chan.org"
        rv = self.app.get(url_for('spider', url=url))
```

```
    assert b'Please wait while the spider scrapes the target website...' in rv.data
    url = "invalidsite.com"
    rv = self.app.get(url_for('spider', url=url), follow_redirects=True)
    assert b'Scrape target website' in rv.data
```

The fourth test attempts to access the report() method to generate a report, first while signed out, then while signed in with valid credentials. While signed out, the signin.html page should open, and while signed in, the report.html page should open.

```
def test_4_report(self):
  with app.app.app_context():
    self.signout()
    rv = self.app.get(url_for('report'), follow_redirects=True)
    assert b'Please sign in' in rv.data
    self.signin('admin', 'admin123')
    rv = self.app.get(url_for('report'))
    assert b'Catch report' in rv.data
```

The fifth test signs the user out and attempts to access the advancedReport(table,field,param) method, and asserts that the signin.html page should open. The test then signs the user in with valid credentials and the report.html page should open. After running the test script, each test passed in 5.039 seconds with no failed tests.

```
Ran 5 tests in 5.039s
OK
```

These tests show that the developed product works as intended.

CHAPTER-8
DEPLOYMENT

8.1 DEPLOYMENT ENVIRONMENT

- **Primary Environment:** The typical deployment environment for this web scraping system is a computer system (desktop, server, or virtual machine) running a standard operating system (Linux, macOS, or Windows) where the Python interpreter can be executed. Unlike a traditional web application, this scraper is generally a command-line script or tool and does **not** require a persistent web server environment like Apache or IIS for its core operation.
- **Execution Context:** The script runs as a process, performs its tasks (fetching, parsing, saving data), and then typically exits. It does not continuously listen for incoming web requests.
- **Network Access:** The environment must have reliable outbound internet access to connect to the target websites for scraping.

8.2 HOSTING OPTIONS

- **Local Machine:** The simplest option is running the script directly on a developer's or end-user's computer for on-demand scraping tasks.
- **On-Premise Server:** Deploying the script on a dedicated server within the college/organization. This is suitable for running scheduled scraping tasks using OS-level schedulers (cron on Linux/macOS, Task Scheduler on Windows) and for centralizing data output.
- **Cloud Hosting (IaaS - VM):** Utilizing a cloud virtual machine provider (e.g., AWS EC2, Google Compute Engine, Azure Virtual Machines). This offers flexibility, scalability (adjusting CPU/RAM/storage), potentially better network connectivity, and allows for robust scheduling and integration with other cloud services.
- **Cloud Hosting (PaaS/Serverless - Advanced):** For more advanced or event-driven scenarios, the script could potentially be adapted and packaged for deployment on serverless platforms (e.g., AWS Lambda, Google Cloud Functions). This often requires modifications for handling execution time limits and state management but can be cost-effective for specific scheduling needs.

8.3 SOFTWARE & HARDWARE REQUIREMENTS

8.3.1 SOFTWARE REQUIREMENTS

- **Operating System:** Linux, macOS, or Windows.
- **Python:** A compatible version (e.g., Python 3.8+) with pip.
- **Python Libraries:** The specific libraries listed in the project's requirements.txt file (must include beautifulsoup4, requests, potentially lxml, etc.).
- **Version Control (Recommended):** Git client for obtaining and managing code updates.
- **Scheduling Tools (Optional):** cron (Linux/macOS) or Task Scheduler (Windows) for automated runs.
- **Not Required Typically:** Web server software (Apache, IIS), Web framework (Django), Database server (MySQL) - unless the scraper is specifically designed as part of a larger web application or to write directly into a specific database.

8.3.1 HARDWARE REQUIREMENTS:

- **Processor (CPU):** Standard modern CPU capable of running Python efficiently.
- **Memory (RAM):** Sufficient RAM (e.g., minimum 1-2 GB, more recommended depending on data volume and complexity) for the Python process, libraries, and data handling.
- **Disk Space:** Enough storage for the OS, Python installation, libraries, project code, and significantly, for storing the potentially large output data files generated by scraping. (Refer to Chapter 2 [2] for specific baseline recommendations if available, e.g., 20GB disk).
- **Network:** Stable internet connection.

8.4 DEPLOYMENT PROCESS

- **Prepare Server/Machine:** Ensure the target machine meets the software/hardware requirements, including Python installation and network access.
- **Obtain Code:** Clone the Git repository or copy the project files onto the server/machine.
- **Set Up Environment:** Navigate to the project directory. Create and activate a Python virtual environment (python -m venv venv, source venv/bin/activate or .\venv\Scripts\activate). This isolates dependencies, a standard practice emphasized even for scripting tasks.
- **Install Dependencies:** Run pip install -r requirements.txt within the active virtual environment.

- **Configure Scraper:** Adjust configuration settings as needed (target URLs, CSS selectors/XPath expressions, output file path, request delays) either by editing configuration files (config.py, .env), passing command-line arguments, or modifying script variables directly.
- **Test Execution:** Run the main scraper script manually from the command line to ensure it fetches, parses, and saves data correctly: python main_scraper.py [arguments]. Check log outputs and the generated data file.
- **Schedule Execution (Optional):** If automated runs are needed, configure a cron job or a Task Scheduler task to execute the script using the Python interpreter located within the virtual environment. Ensure the job has appropriate permissions and logs output/errors.(No URL/SSL/Firewall rules specifically for the script, but host machine firewall needs to allow outbound HTTPS/HTTP traffic).

8.5 MAINTENANCE

- **Code Management:** Use version control (Git) to track changes to the scraper code, especially updates to selectors when target websites change structure (a frequent necessity highlighted in web scraping literature).
- **Dependency Updates:** Periodically review and update libraries within the virtual environment (pip list --outdated, pip install -U <package>) and update requirements.txt (pip freeze > requirements.txt).
- **Monitoring:** Regularly check execution logs (especially for scheduled tasks) to identify errors (network issues, parsing failures, changes in website structure blocking data extraction). Monitor resource usage (disk space for output files, CPU/RAM during runs) on the host machine. Periodically validate the quality and structure of the scraped data.
- **Target Website Monitoring:** The most critical maintenance involves monitoring the target websites for structural changes that break the scraper's selectors. This often requires manually inspecting the site and updating the code accordingly.
- **OS/Python Security:** Keep the underlying operating system and the Python interpreter installation updated with security patches.
- **(Database Backups):** Only required if the scraper is configured to write directly to a database. Regular backups of the output data files (CSVs, JSONs) themselves are recommended.

8.6 Deployment Process

- **Prepare Server/Machine:** Ensure the target machine meets the software/hardware requirements, including Python installation and network access.
- **Obtain Code:** Clone the Git repository or copy the project files onto the server/machine.
- **Set Up Environment:** Navigate to the project directory.Create and activate a Python virtual environment (python -m venv venv, source venv/bin/activate or .\venv\Scripts\activate). This isolates dependencies, a standard practice emphasized even for scripting tasks.

CHAPTER-9
RESULT AND OUTPUT SCREENS

9.1 HOME PAGE

Figure 9.1: Home Page

9.2 FILES

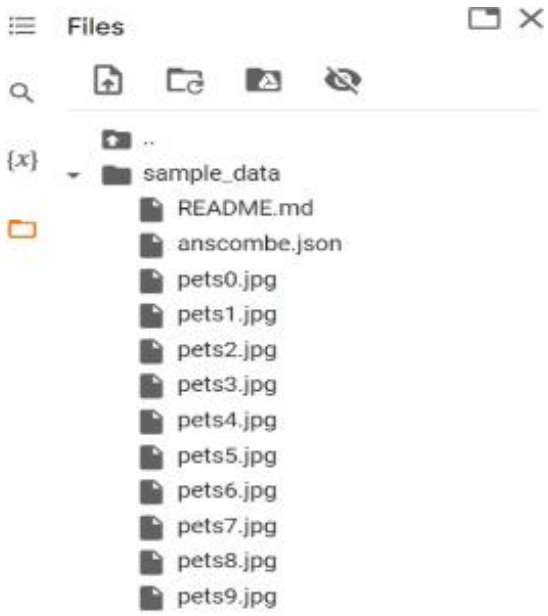


Figure 9.2: Files

CHAPTER-10
CONCLUSION AND FUTURE WORK

10.1 CONCLUSION

The web scraping system developed in this project successfully demonstrates an efficient and automated solution for extracting targeted data from websites using Python and the Beautiful Soup library. By programmatically fetching web pages and parsing their underlying HTML structure, the system effectively overcomes the limitations of manual data collection, such as slowness, scalability issues, and proneness to error. The implementation provides structured data output (e.g., in CSV or JSON format), making the collected information readily available for analysis, reporting, or integration into other applications. This project highlights the practical utility of Beautiful Soup for navigating and extracting information from static or server-rendered web pages, offering a significant improvement in efficiency for data acquisition tasks compared to traditional manual methods.

10.2 FUTURE WORK

While the current implementation effectively scrapes data using Beautiful Soup, several areas offer potential for future enhancement and development, drawing inspiration from broader web scraping techniques discussed in literature like "Web Scraping with Python, 2nd Edition":

REFERENCES

JOURNALS / RESEARCH PAPERS / THESES

1. Gallagher, Evan (2018) *Scraping Websites for Law Enforcement*. BSc Hons Computer Science Thesis, Ulster University. (Provides context on web scraping applications, defines web scrapers, and discusses implementation using frameworks like Scrapy).

BOOKS

2. Mitchell, Ryan (2018) *Web Scraping with Python: Collecting More Data from the Modern Web*. 2nd Edition, O'Reilly Media. (Comprehensive guide covering Python web scraping mechanics, Beautiful Soup, handling various data sources, advanced techniques like interacting with forms/JavaScript/APIs, and ethical considerations).

WEBSITES

3. Beautiful Soup Documentation: *Official documentation for the Beautiful Soup library (bs4)*. Available at: https://www.crummy.com/software/BeautifulSoup/bs4/doc/
4. Requests: HTTP for Humans™: *Official documentation for the Python Requests library*. Available at: https://requests.readthedocs.io/en/latest/
5. Python.org: *The official website for the Python programming language*. Available at: https://www.python.org/
6. The Web Robots Pages: *Information about the Robots Exclusion Protocol (robots.txt)*. Available at: https://www.robotstxt.org/

PROJECT SUMMARY

About Project

Title of the project	Design and implementation of web scrapping using beautiful soup
Semester	8th
Members	4
Team Leader	Nidhi Rani
Describe role of every member in the project	**Nidhi Rani:** Backend Development **Manish Singh Shayam:** Frontend Development **Praful Solanki:** Testing **Md Hedayatullah:** Documentation
What is the motivation for selecting this project?	The project aims to save significant time and manual effort by automatically extracting targeted data from websites using Python and Beautiful Soup.
Project Type (Desktop Application, Web Application, Mobile App, Web)	Web Application

Tools &Technologies

Programming language used	Python
IDE used (with version)	VS code (1.98)
Front End Technologies (with version, wherever Applicable)	Python
Back End Technologies (with version, wherever applicable)	Python, Database
Database used (with version)	My Sql

Software Design& Coding

Is prototype of the software developed?	NA

SDLC model followed (Waterfall, Agile, Spiral etc.)	Agile
Why above SDLC model is followed?	SDLC models are frameworks that guide development of software applications from conception to deployment and maintenance.
Justify that the SDLC model mentioned above is followed in the project.	Changing Requirements, Client Involvement, Iterative Development, Early Delivery of Value, Adaptability, Collaborate Team Environment, Continuous Testing, Risk Management, Customer Satisfaction.
Software Design approach followed (Functional or Object Oriented)	Functional
Name the diagrams developed (according to the Design approach followed)	Use case Diagram ER Diagram
In case Object Oriented approach is followed, which of the OOPS principles are covered in design?	
No. of Tiers (example 3-tier)	3-tier
Total no. of front end pages	3
Front end validations applied (Yes / No)	Yes
Session management done (in case of web applications)	Yes
Is application browser compatible (in case of web applications)	Yes
Exception handling done (Yes / No)	No
Commenting done in code (Yes / No)	Yes

Naming convention followed (Yes / No)	yes
What difficulties faced during deployment of project?	
Total no. of Use-case s	1
Give titles of Use-cases	Use-case Diagram

Project Requirements

MVC architecture followed (Yes / No)	Yes
If yes, write the name of MVC architecture followed (MVC-1, MVC-2)	MVC-2
Design Pattern used (Yes / No)	No
If yes, write the name of Design Pattern used	-
Interface type (CLI / GUI)	CLI
No. of Actors	-
Name of Actors	-
Total no. of Functional Requirements	5
List few important non-Functional Requirements	Correctness, Flexibility, Reliability and Maintainability.

Testing

Which testing is performed? (Manual or Automation)	Manual
Is Beta testing done for this project?	No

Write project narrative covering above mentioned points

Developed a web scraping system using Python and the Beautiful Soup library to automate data extraction from target websites. Enhanced data acquisition efficiency by programmatically parsing HTML and extracting specific information, overcoming the limitations of manual collection. Focused on implementing robust fetching and parsing logic, selector-based data identification, and structured output generation (e.g., CSV/JSON) suitable for analysis.

Manish Singh Shayam	0187CS211099	Guide Signature
Praful Solanki	0187CS211119	(Prof. Pankaj Savita)
Nidhi Rani	0187CS211108	
Md Hedayatullah	0187CS211100	

APPENDIX-1 GLOSSARY OF TERMS

C

CLI Command Line Interface; A text-based user interface used to view and manage computer files.

CSS Cascading Style Sheets; A style sheet language used to control presentation of a document written in HTML or XML.

E

ER Entity Relationship; A high-level conceptual data model diagram that depicts the entities and relationships in an information system.

G

GUI Graphical User Interface; A user interface that involves visual icons, menus and graphics as opposed to text-based interfaces.

H

HTML Hypertext Markup Language; Standard markup language for documents designed to be displayed in a web browser.

M

MVT Model-View-Template; A software design pattern used to structure web applications that separates an application into three interconnected parts.

MYSQL An open source relational database management system based on SQL queries.

O

ORM Object-Relational Mapping; Programming technique to convert incompatible type systems in object-oriented programming languages into relational database schemas.

U

UML

Unified Modeling Language; Standardized modeling language enabling developers to specify, visualize, and document software systems..

V

VS CODE

Visual Studio Code; Lightweight code editor for building and debugging modern web and cloud applications.